CallHub – Phone Directory Management System

Assignment 2 Module-A Report

GitHub GitHub Repository
Video Video Demonstration

1 Project Overview

CallHub is a Phone Directory Management System for IIT Gandhinagar, where contact information is scattered across spreadsheets and paper directories. This report covers **Module A**, the indexing and data structure layer.

Module A implements a lightweight in-memory DBMS using a **B+ Tree** (order 6) as the core indexing engine, covering all 14 tables from the Assignment 1 schema. It includes a BruteForceDB baseline, automated benchmarking, and Graphviz visualisation.

2 Methodology

- Implement all B+ Tree operations following the instructor's boilerplate method signatures exactly
- Maintain balance through proactive node splitting on insert and borrowing/merging on delete
- Use a leaf-node linked list for efficient range queries
- Compare B+ Tree against BruteForceDB at 10 dataset sizes (N = 100 to 4600) with fixed random seed 42
- Measure time and peak memory using `tracemalloc`
- Visualise the tree using `graphviz.Digraph`

3 Functionality

- Full B+ Tree with all boilerplate methods: `_insert_non_full`, `_split_child`, `_delete`, `_fill_child`, `_borrow_from_prev`, `_borrow_from_next`, `_merge`, `_add_nodes`, `_add_edges`
- Insert, exact search O(log n), range query via leaf linked list, delete with underflow handling, update in place
- Value storage – leaf nodes hold full CallHub Member record dictionaries

- Schema-aware Table class and two-level DatabaseManager supporting all 14 CallHub tables
- PerformanceAnalyzer benchmarking 6 metrics across 10 dataset sizes
- Graphviz visualisation: blue internal nodes, green leaf nodes, dashed green next-leaf arrows

4 Approach

- Followed the instructor's boilerplate structure exactly including `BPlusTreeNode(order, is_leaf)` and `is_full()`
- Proactive split on insert, proactive borrow/merge on delete
- Leaf linked list maintained correctly through every split and merge
- All 20 members, 20 departments, 20 roles, 5 labs, and 5 office rooms from Assignment 1 loaded in the notebook
- B+ Tree always benchmarked against BruteForceDB under identical workloads

5 Implementation Evidence (Images/Graphs)

5.1 Operations Demo

The figure below shows insert, search, delete, and range query running on the CallHub Member table with all 20 members loaded from Assignment 1 sample data.

5.2 Operations Demo

```
Inserting members...
  insert(1) → (True, 1)
  insert(2) → (True, 2)
  insert(3) → (True, 3)
  insert(4) → (True, 4)
  insert(5) → (True, 5)
  insert(6) → (True, 6)
  insert(7) → (True, 7)
  insert(8) → (True, 8)
  insert(9) → (True, 9)
  insert(10) → (True, 10)
  insert(11) → (True, 11)
  insert(12) → (True, 12)
  insert(13) → (True, 13)
  insert(14) → (True, 14)
  insert(15) → (True, 15)
  insert(16) → (True, 16)
  insert(17) → (True, 17)
  insert(18) → (True, 18)
  insert(19) → (True, 19)
  insert(20) → (True, 20)

Total members: 20
Tree height  : 3
```

Figure 1: (a) Insert – 20 members

```
get(member_id=6):
{'member_id': 6, 'member_name': 'Neha Gupta', 'iit_email': 'neha@org.in', 'primary_phone': '9000000006', 'dob': '2000-06-06', 'department_
id': 6, 'is_at_campus': True, 'join_date': '2022-01-01'}

get(member_id=14):
{'member_id': 14, 'member_name': 'Kavita Joshi', 'iit_email': 'kavita@org.in', 'primary_phone': '9000000014', 'dob': '1990-04-14', 'depart
ment_id': 4, 'is_at_campus': True, 'join_date': '2012-01-01'}

get(member_id=999) — not found:
None
```

Figure 2: (b) Exact Search

```
Update member_id=3 (Rahul Mehta — change phone and department):
Before: {'member_id': 3, 'member_name': 'Rahul Mehta', 'iit_email': 'rahul@org.in', 'primary_phone': '9000000003', 'dob': '2001-03-03', 'd
epartment_id': 3, 'is_at_campus': True, 'join_date': '2023-01-01'}
update() → (True, 'Record updated successfully')
After  : {'member_id': 3, 'member_name': 'Rahul Mehta', 'iit_email': 'rahul@org.in', 'primary_phone': '9777777003', 'dob': '2001-03-03',
'department_id': 1, 'is_at_campus': True, 'join_date': '2023-01-01'}
```

Figure 3: (c) Update

```
range_query(5, 10):
  [5] Karan Patel      | dept_id=5 | 9000000005
  [6] Neha Gupta       | dept_id=6 | 9000000006
  [7] Arjun Nair       | dept_id=7 | 9000000007
  [8] Pooja Singh      | dept_id=8 | 9000000008
  [9] Rohit Das        | dept_id=9 | 9000000009
  [10] Ananya Rao       | dept_id=10 | 9000000010
```

Figure 4: (d) Range Query

B+ Tree core operations on CallHub Member table
B+ Tree core operations – insert, search, update, delete, range query output from report.ipynb

5.3 B+ Tree Visualisation

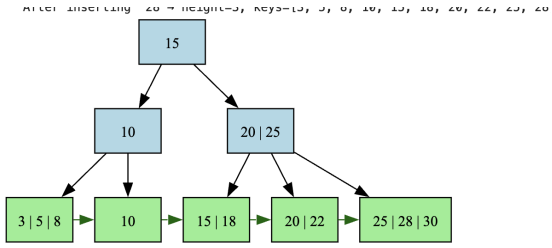

Figure 5: (a) Demo tree – node splitting

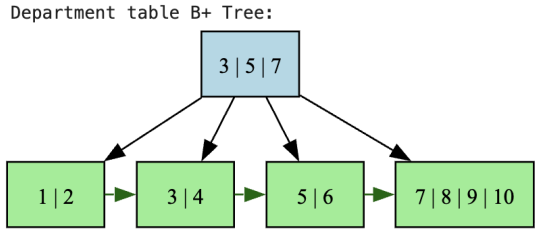

Figure 6: (b) Department table tree

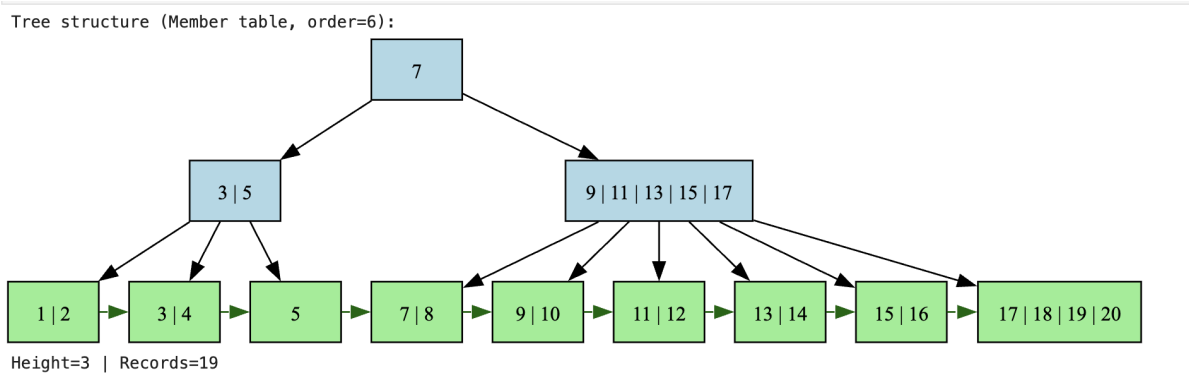

Figure 7: (c) Member table – 20 records, height=3

B+ Tree visualisations. Blue = internal nodes, Green = leaf nodes, Arrows = leaf linked list

5.4 Benchmark Results

Summary – Averages across all 10 dataset sizes:

Operation	B+ Tree (ms)	Brute Force (ms)	Speedup	Winner
Insert	6.19	104.80	16.94×	B+ Tree
Search	0.32	6.07	18.74×	B+ Tree
Range Query	0.05	0.09	1.72×	B+ Tree
Delete	0.32	4.79	15.06×	B+ Tree
Random Mix	5.52	67.76	12.27×	B+ Tree
Memory (KB)	1322.4	1062.6	—	Brute Force

Interpretation: Search (18.74×) and insertion (16.94×) show the largest gains because BruteForce O(n) scan grows fast while B+ Tree stays O(log n). Range query gain is smaller (1.72×) since the range covers 25% of data. Memory is higher in B+ Tree due to internal node and pointer overhead – the standard time-space trade-off for indexed structures.

5.5 Benchmark Results

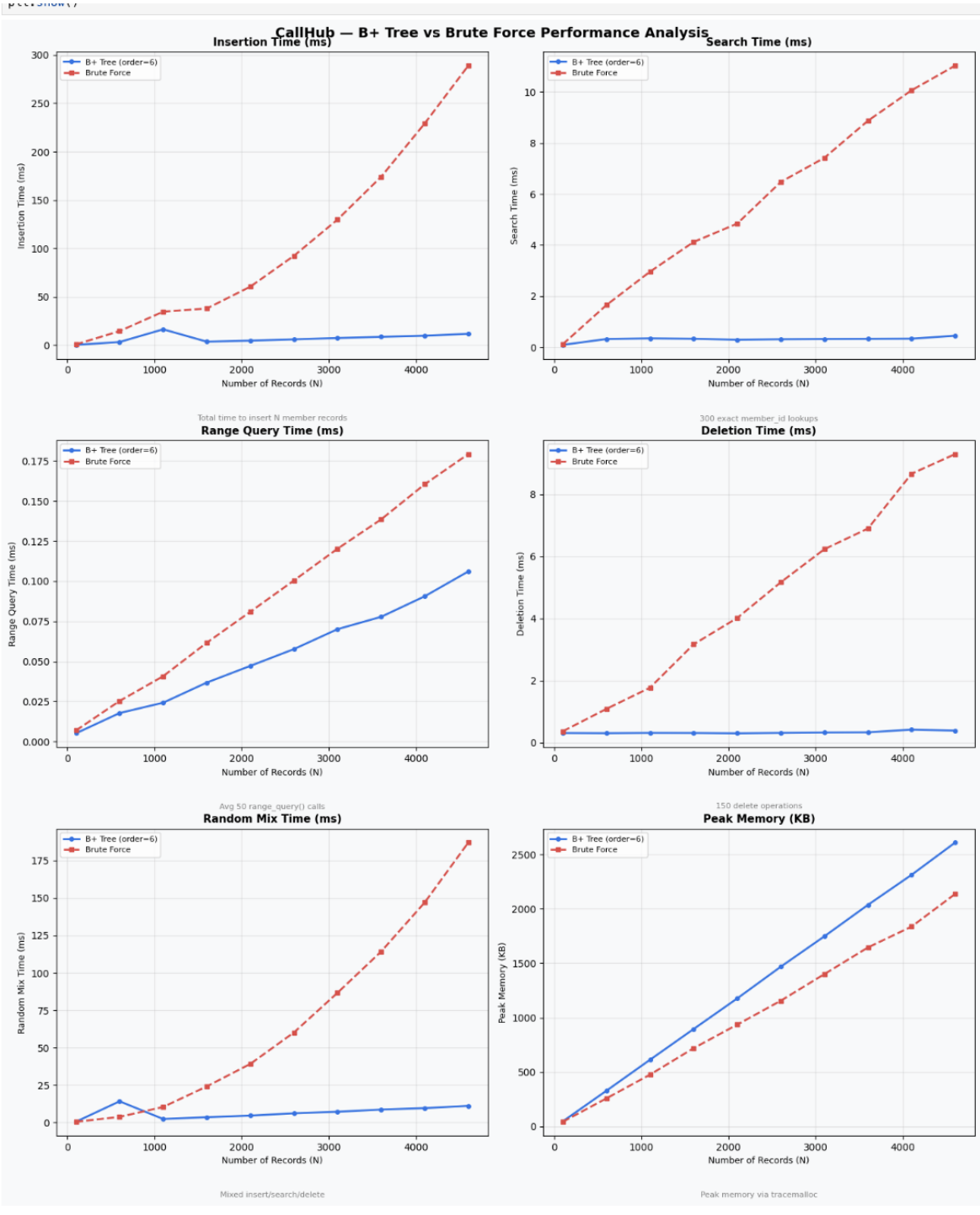

Figure 8: B+ Tree vs Brute Force – performance plots across N=100 to N=4600

```
======================================================================
  PERFORMANCE SUMMARY  (avg over all N sizes)  [time in ms]
======================================================================
Op                   B+ Tree    Brute Force    Speedup
----------------------------------------------------------------
Insert                7.1203       106.2381     14.92x
Search                0.3095         5.7522     18.59x
Range                 0.0533         0.0914      1.71x
Delete                0.3347         4.6687     13.95x
Random                6.9016        67.3255      9.76x
Memory(KB)            1323.9         1061.1 (see note)
======================================================================
Note: B+ Tree uses more memory due to pointer overhead (children, next).
```

Figure 9: Performance summary – averages across all dataset sizes

6 Tools and External Sources Disclosure

Tools: Python 3.11, Graphviz, Matplotlib, Jupyter Notebook, Git/GitHub, `tracemalloc` (Python stdlib).

External Sources: Official documentation for Python, Graphviz, and Matplotlib. B+ Tree concept reference: YouTube playlist provided by the instructor. No external B+ Tree libraries are used, implemented entirely from scratch.

7 Team Member Roles

Team Member	Roll No	Contribution
Sanjay	23110144	B+ Tree core – insert with node splitting, delete with borrowing and merging, exact search, range query via leaf linked list, update, and all internal helper methods. Notebook demo and sample data loading.
Thushar	24110393	Table class, DatabaseManager, BruteForceDB, PerformanceAnalyzer, benchmark execution, Matplotlib graphs, Graphviz visualisation, loading all 14 CallHub tables, report, and video.

8 Conclusion

Module A delivers a complete B+ Tree DBMS engine from scratch supporting all 14 CallHub tables, with measured speedups of up to 18.74× over BruteForce on search. The implementation follows the instructor's boilerplate exactly. Full evidence is in **report.ipynb** and the video demonstration.